```
          ticketsRemaining.toString()
        e.dataset.concertSoldOutValue = (
          ticketsRemaining === 0
        ).toString()
      }
    })
  })
 }
}
```

Three changes were made here. First, at the top of the file, there's a new interface ConcertRemainingData that contains the concertId and the ticketsRemaining. Strictly speaking, this isn't necessary, but TypeScript likes to know this information, and we do have it. Second, the createSubscription changes slightly to expect concerts as the top-level key in the incoming JSON when data is received. And third, the updateData method changes too. For each entry in the data, it checks all the concert targets, and if the IDs match, it updates the tickets remaining and sold-out information in the element dataset. (For performance reasons, you'd probably want the concertTargets to already be indexed to avoid the inner loop, but it's not a significant problem yet.)

And we have one more small change to make to ensure the display actually updates.

To do this, we need to change the name of the method in the Stimulus ConcertController to watch the tickets remaining value, like this:

chapter_09/05/app/javascript/controllers/concert_controller.ts

```
import { Controller } from "@hotwired/stimulus"

export default class ConcertController extends Controller {
  static targets = ["tickets"]
  ticketsTarget: HTMLElement

  static values = { id: Number, soldOut: Boolean, ticketsRemaining: Number }
  soldOutValue: boolean
  ticketsRemainingValue: number

  ticketsRemainingValueChanged(): void {
```